CRDT

Tipo de Dado Replicado e Livre de Conflitos (Conflict-free Replicated Data Type)

Edson (Edinho)

Python Brasil 2026

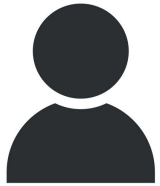
Agenda

1. O que são e para que servem?
2. Teorema CAP
3. O fundamento por trás dos CRDTs
4. Um primeiro exemplo
5. Todos os CRDTs conhecidos
6. Mais exemplos

Agenda

1. **O que são e para que servem?**
2. Teorema CAP
3. O fundamento por trás dos CRDTs
4. Um primeiro exemplo
5. Todos os CRDTs conhecidos
6. Mais exemplos

O que são e para que servem?



Usuária A



Usuária B

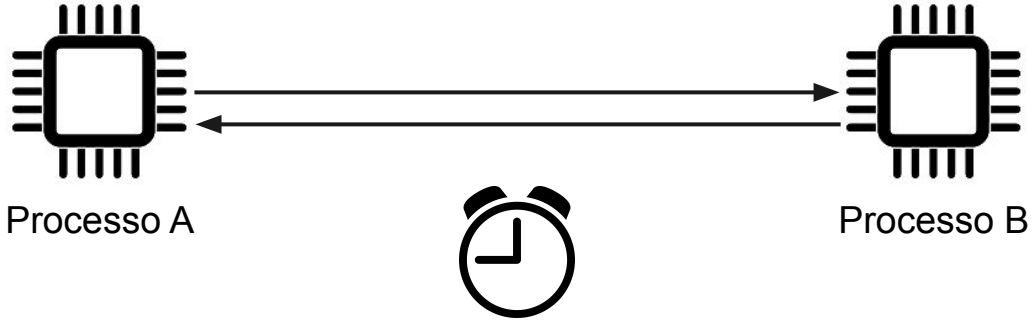


Comp. A

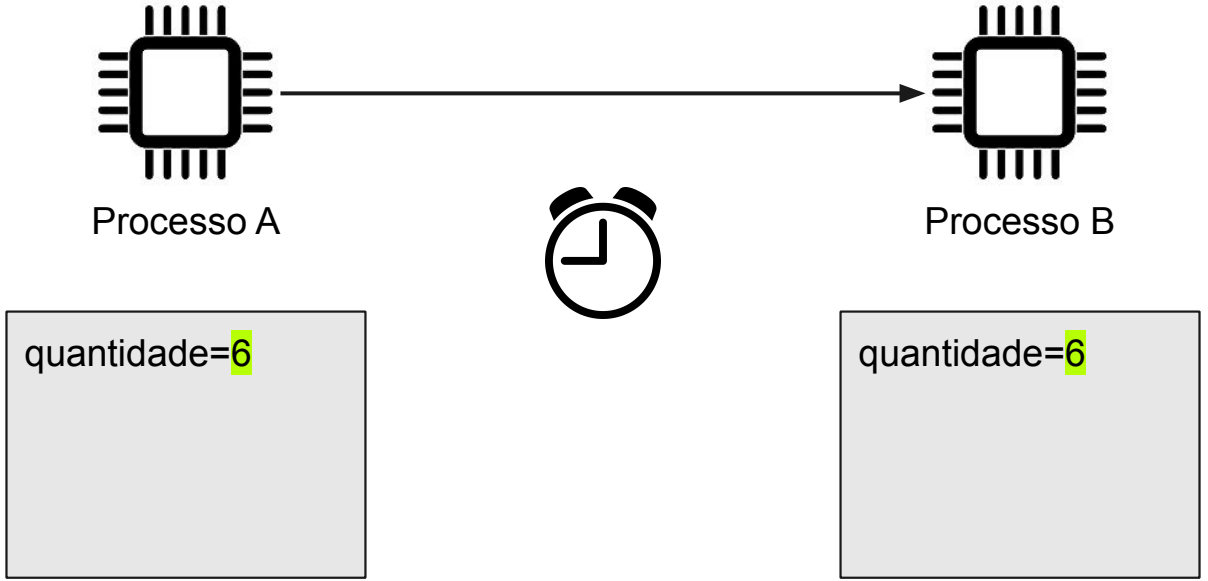


Comp. B

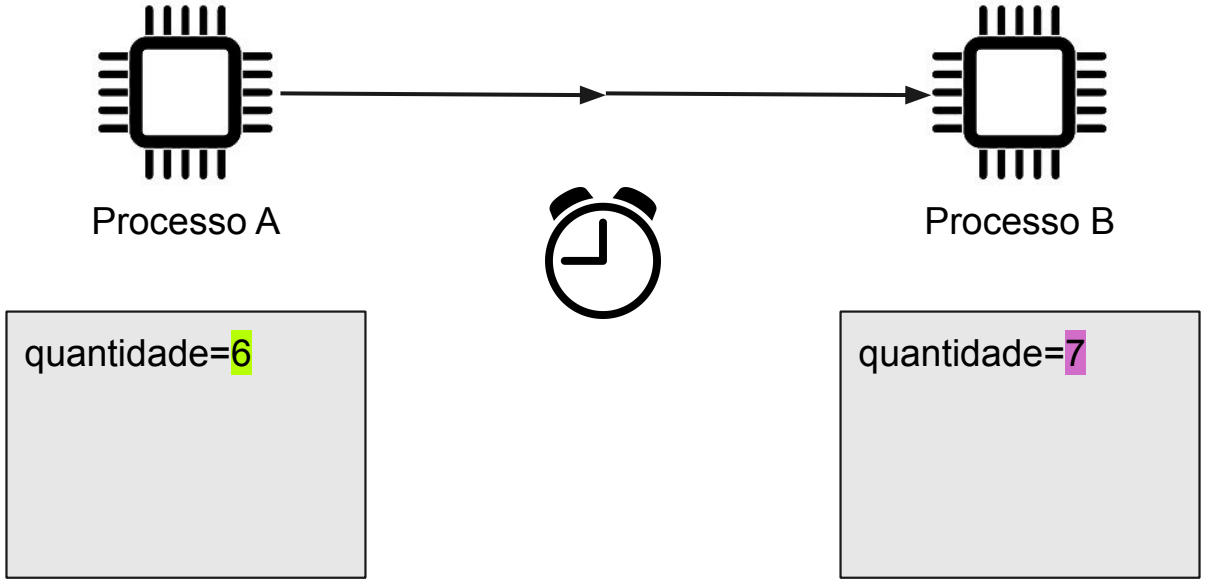
O que são e para que servem?



O que são e para que servem?



O que são e para que servem?





An error has occured.We dont even know what is it.So cant fix it ,
and you have restart your computer.By the way if you restart your ,
computer you will lose any unsaved information in all open applications.
In the other hand you dont have any other option :)

Press Enter to return to Windows(It wont work), or

Press CTRL+ALT+DEL to restart your computer.

Error : 0E : 016F : BFF9B3D4

Agenda

1. **O que são e para que servem?**
2. Teorema CAP
3. O fundamento por trás dos CRDTs
4. Um primeiro exemplo
5. Todos os CRDTs conhecidos
6. Mais exemplos

Agenda

1. O que são e para que servem?
2. **Teorema CAP**
3. O fundamento por trás dos CRDTs
4. Um primeiro exemplo
5. Todos os CRDTs conhecidos
6. Mais exemplos

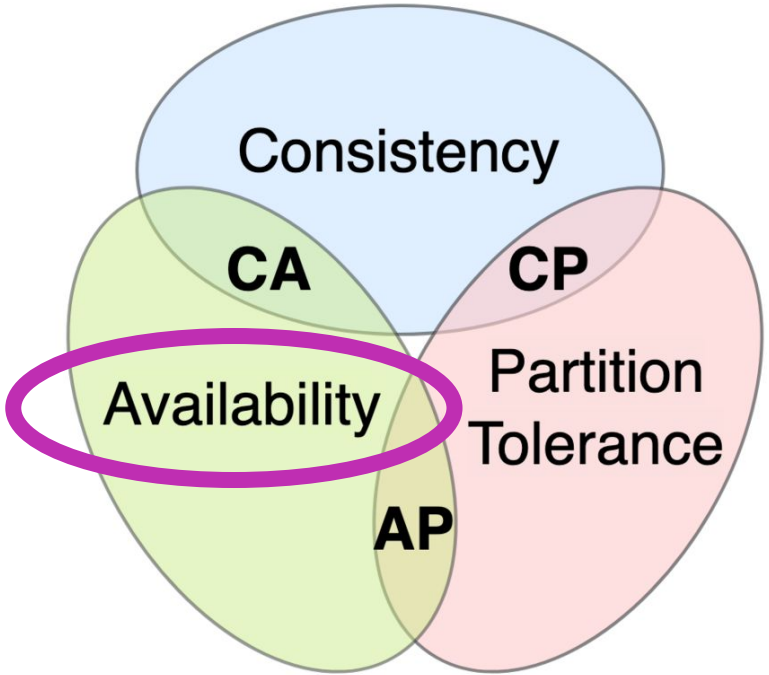
Teorema CAP

Consistency	Consistência
Availability	Disponibilidade
Partition Tolerance	Tolerância a Falha de Rede

Teorema CAP

Consistency	Consistência
Availability	Disponibilidade
Partition Tolerance	Tolerância a Falha de Rede

Teorema CAP



Agenda

1. O que são e para que servem?
2. **Teorema CAP**
3. O fundamento por trás dos CRDTs
4. Um primeiro exemplo
5. Todos os CRDTs conhecidos
6. Mais exemplos

Agenda

1. O que são e para que servem?
2. Teorema CAP
3. **O fundamento por trás dos CRDTs**
4. Um primeiro exemplo
5. Todos os CRDTs conhecidos
6. Mais exemplos

O fundamento por trás dos CRDTs

- É permitido atualizar qualquer réplica de forma independente.
- Existe um algoritmo que resolve automaticamente quaisquer inconsistências.
- É garantida a convergência de todas as réplicas para o mesmo estado.

Shapiro et al. (2011). Conflict-free Replicated Data Types.

O fundamento por trás dos CRDTs

- É permitido atualizar qualquer réplica de forma **independente**.
- Existe um algoritmo que resolve automaticamente **quaisquer** inconsistências.
- É **garantida** a convergência de **todas** as réplicas para o mesmo estado.

Shapiro et al. (2011). Conflict-free Replicated Data Types.

O fundamento por trás dos CRDTs

```
class CRDT:
    def merge(self, other: Self) -> Self:
        ...
```

Shapiro et al. (2011). Conflict-free Replicated Data Types.

O fundamento por trás dos CRDTs

```
# comutatividade
merge(a, b) == merge(b, a)
```

Shapiro et al. (2011). Conflict-free Replicated Data Types.

O fundamento por trás dos CRDTs

```
# comutatividade
merge(a, b) == merge(b, a)

# associatividade
merge(a, merge(b, c)) == merge(merge(a, b), c)
```

Shapiro et al. (2011). Conflict-free Replicated Data Types.

O fundamento por trás dos CRDTs

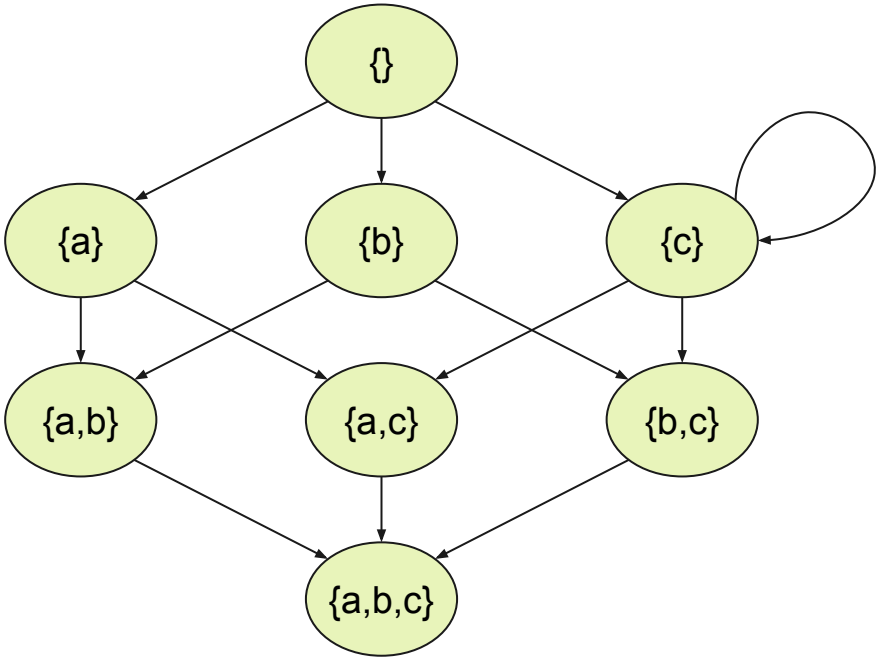
```
# comutatividade
merge(a, b) == merge(b, a)

# associatividade
merge(a, merge(b, c)) == merge(merge(a, b), c)

# idempotência
merge(a, a) == a
```

Shapiro et al. (2011). Conflict-free Replicated Data Types.

O fundamento por trás dos CRDTs



Agenda

1. O que são e para que servem?
2. Teorema CAP
3. **O fundamento por trás dos CRDTs**
4. Um primeiro exemplo
5. Todos os CRDTs conhecidos
6. Mais exemplos

Agenda

1. O que são e para que servem?
2. Teorema CAP
3. O fundamento por trás dos CRDTs
4. **Um primeiro exemplo**
5. Todos os CRDTs conhecidos
6. Mais exemplos

Um primeiro exemplo

G-Counter (Grow-only Counter)

É um contador crescente.

Cada réplica pode incrementar o contador como quiser.

Um primeiro exemplo

G-Counter

```
class GCounter:
    def __init__(self, replica_id):
        self.replica_id = replica_id
        self.contador = defaultdict(int)
```

Um primeiro exemplo

G-Counter

```
class GCounter:
    def __init__(self, replica_id):
        self.replica_id = replica_id
        self.contador = defaultdict(int)

    def valor(self):
        return sum(self.contador.values())
```

Um primeiro exemplo

G-Counter

```python
class GCounter:
    def __init__(self, replica_id):
        self.replica_id = replica_id
        self.contador = defaultdict(int)

    def valor(self):
        return sum(self.contador.values())

    def incrementar(self, inc):
        self.contador[self.replica_id] += inc
```

Um primeiro exemplo

G-Counter

```python
class GCounter:
    def __init__(self, replica_id):
        self.replica_id = replica_id
        self.contador = defaultdict(int)

    def valor(self):
        return sum(self.contador.values())

    def incrementar(self, inc):
        self.contador[self.replica_id] += inc

    def merge(self, other):
        for replica_id in other.contador:
            self.contador[replica_id] = max(
                self.contador[replica_id],
                other.contador[replica_id],
            )
```

Um primeiro exemplo

G-Counter

```
>>> a = GCounter('id-a')
>>> b = GCounter('id-b')
```

Um primeiro exemplo

G-Counter

```
>>> a = GCounter('id-a')
>>> b = GCounter('id-b')
>>> print(f'{a.valor()=}    {b.valor()=}')
a.valor()=0    b.valor()=0
```

Um primeiro exemplo

G-Counter

```
>>> a = GCounter('id-a')
>>> b = GCounter('id-b')
>>> print(f'{a.valor()=}     {b.valor()=}')
a.valor()=0    b.valor()=0

>>> a.incrementar(2)
>>> b.incrementar(3)
```

Um primeiro exemplo

G-Counter

```
>>> a = GCounter('id-a')
>>> b = GCounter('id-b')
>>> print(f'{a.valor()=}    {b.valor()=}')
a.valor()=0    b.valor()=0

>>> a.incrementar(2)
>>> b.incrementar(3)
>>> print(f'{a.valor()=}    {b.valor()=}')
a.valor()=2    b.valor()=3
```

Um primeiro exemplo

G-Counter

```
>>> a = GCounter('id-a')
>>> b = GCounter('id-b')
>>> print(f'{a.valor()=}    {b.valor()=}')
a.valor()=0    b.valor()=0

>>> a.incrementar(2)
>>> b.incrementar(3)
>>> print(f'{a.valor()=}    {b.valor()=}')
a.valor()=2    b.valor()=3

>>> a.merge(b)
>>> b.merge(a)
```

Um primeiro exemplo

G-Counter

```
>>> a = GCounter('id-a')
>>> b = GCounter('id-b')
>>> print(f'{a.valor()=}    {b.valor()=}')
a.valor()=0    b.valor()=0

>>> a.incrementar(2)
>>> b.incrementar(3)
>>> print(f'{a.valor()=}    {b.valor()=}')
a.valor()=2    b.valor()=3

>>> a.merge(b)
>>> b.merge(a)
>>> print(f'{a.valor()=}    {b.valor()=}')
a.valor()=5    b.valor()=5
```

Um primeiro exemplo

Agenda

1. O que são e para que servem?
2. Teorema CAP
3. O fundamento por trás dos CRDTs
4. **Um primeiro exemplo**
5. Todos os CRDTs conhecidos
6. Mais exemplos

Agenda

1. O que são e para que servem?
2. Teorema CAP
3. O fundamento por trás dos CRDTs
4. Um primeiro exemplo
5. **Todos os CRDTs conhecidos**
6. Mais exemplos

Todos os CRDTs conhecidos

G-Counter

PN-Counter (positive-negative counter)

G-Set (grow-only set)

2P-Set (two-phase set)

LWW-Element-Set

OR-Set (observed-remove set)

Sequence CRDTs

Agenda

1. O que são e para que servem?
2. Teorema CAP
3. O fundamento por trás dos CRDTs
4. Um primeiro exemplo
5. **Todos os CRDTs conhecidos**
6. Mais exemplos

Agenda

1. O que são e para que servem?
2. Teorema CAP
3. O fundamento por trás dos CRDTs
4. Um primeiro exemplo
5. Todos os CRDTs conhecidos
6. **Mais exemplos**

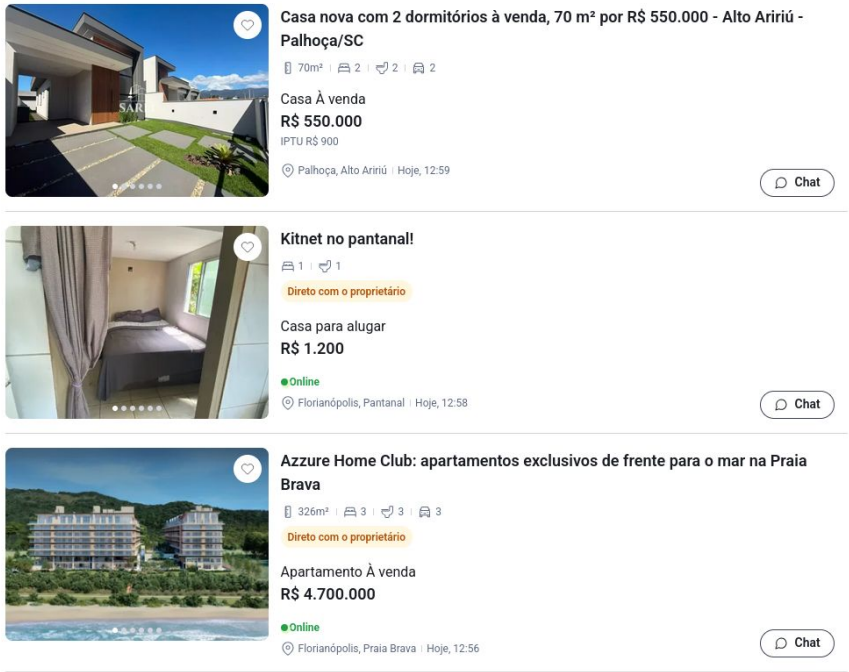
Mais exemplos

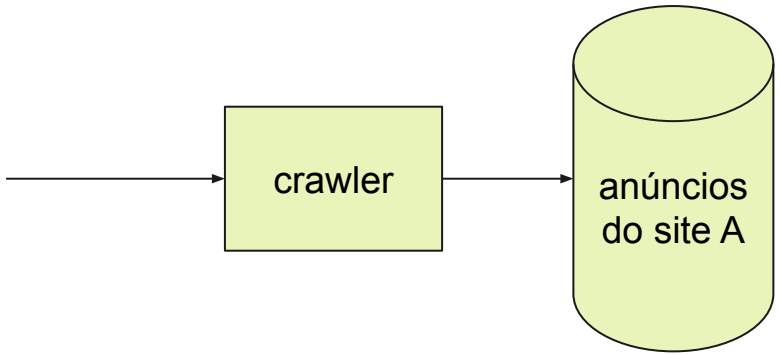
Apple Notes 

Redis 

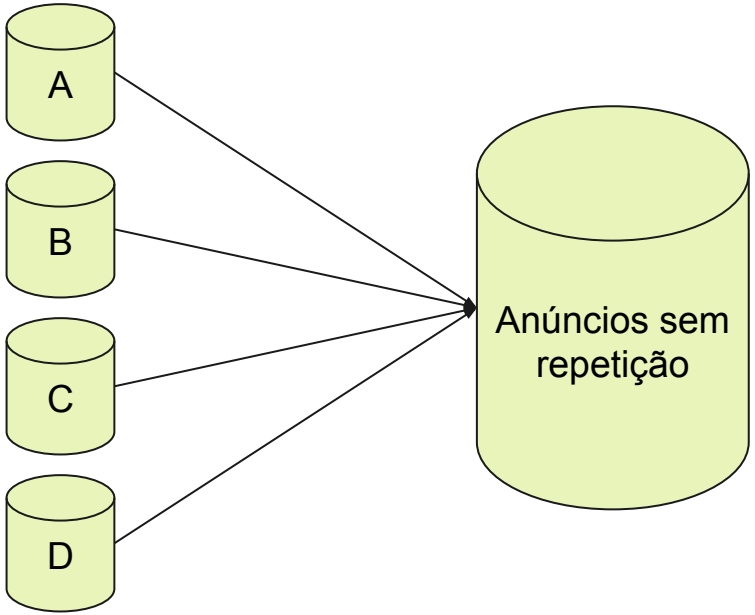
JupyterLab 

Mais exemplos

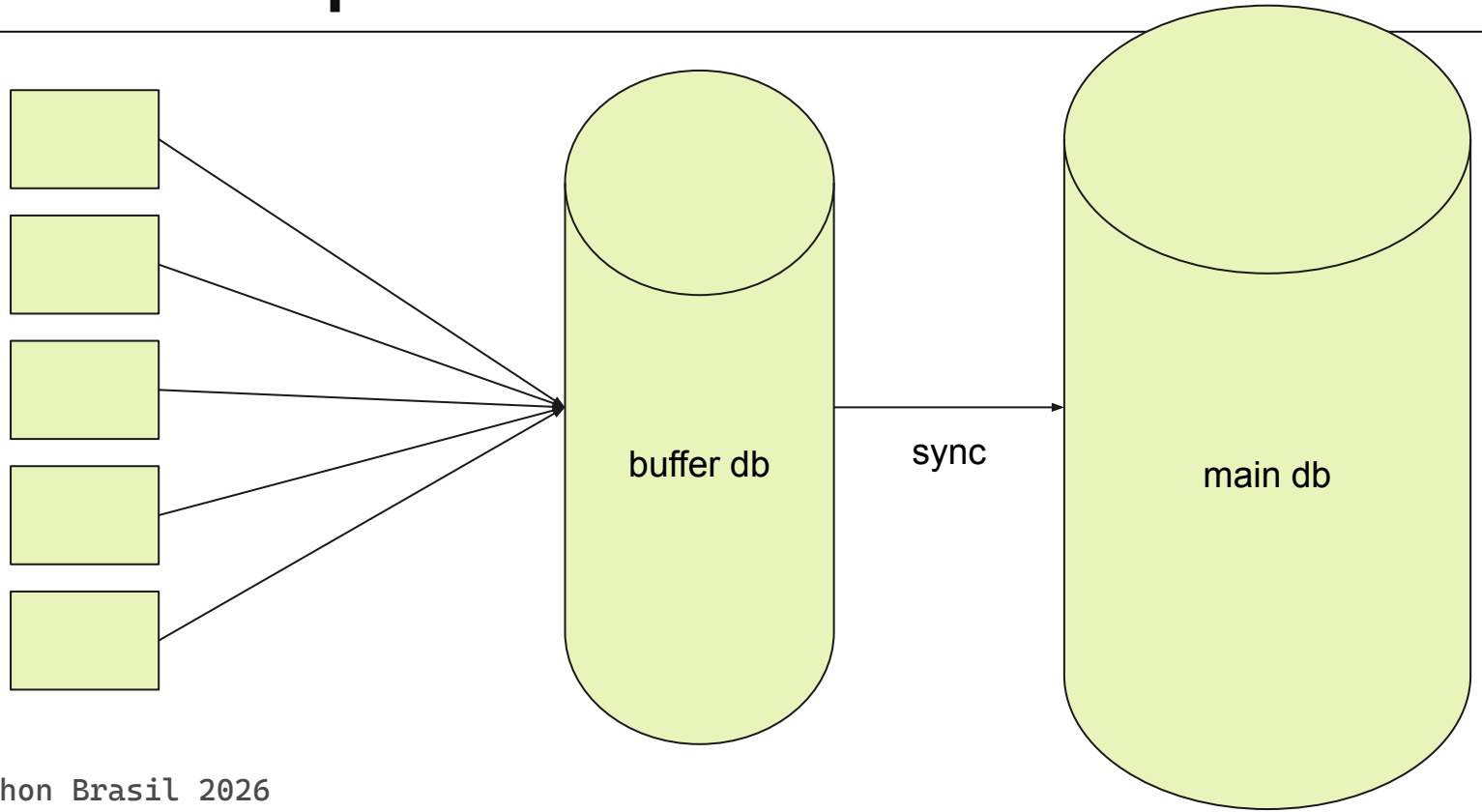




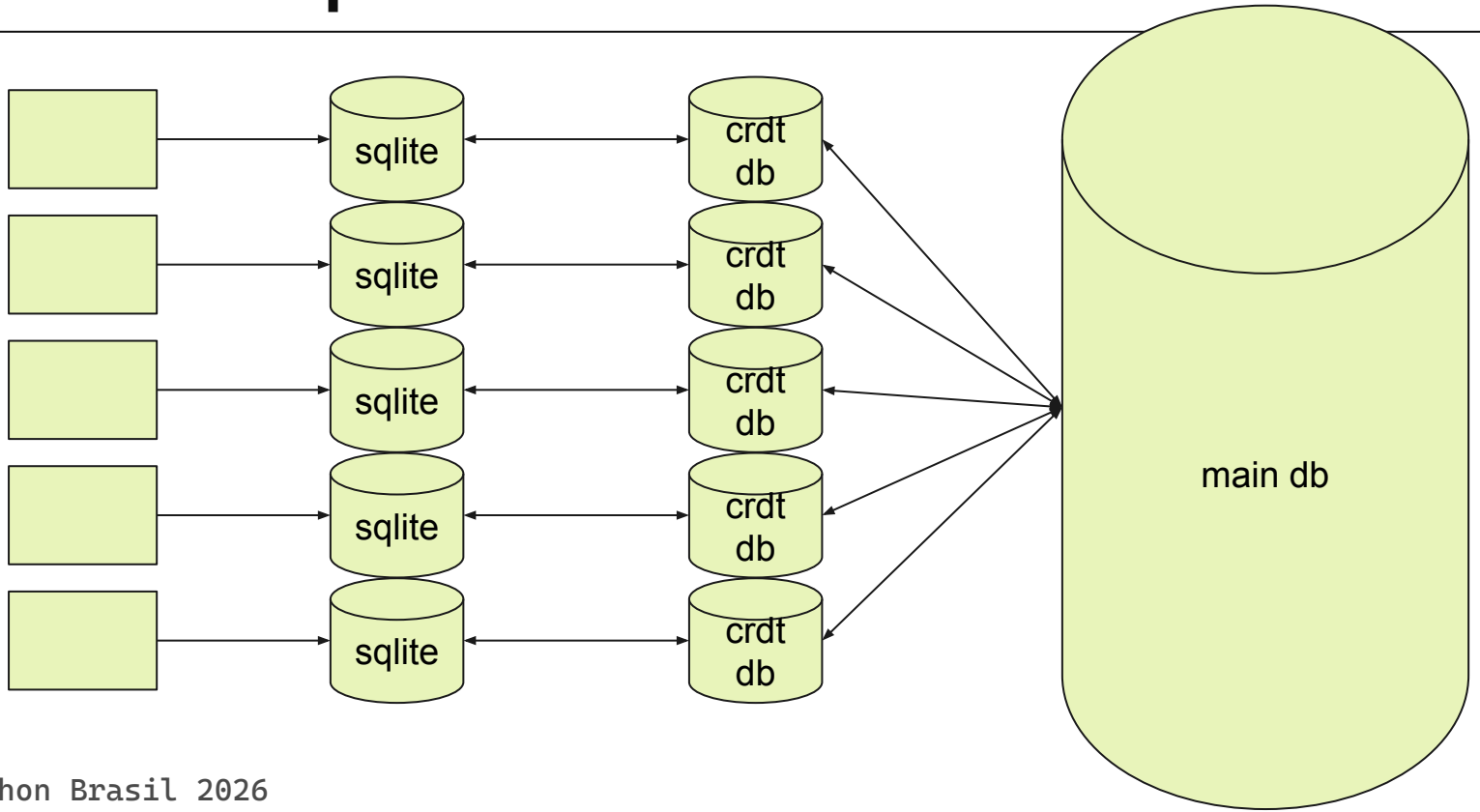
Mais exemplos



Mais exemplos



Mais exemplos



Referências

[1] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, "Conflict-free Replicated Data Types," Inria, Centre Paris-Rocquencourt, Research Report RR-7687, Jul. 2011.
Disponível em: https://hal.science/inria-00609399

https://crdt.tech/